EE114: Introduction to Embedded Systems

Dr. Tao Wu 吴涛 研究员，博导

wutao@shanghaitech.edu.cn

Office: SIST Bldg 3-326

CourseWeb: http://10.15.25.5/

Website: http://small.shanghaitech.edu.cn

Demo Task

- USART DMA 方式编程 （固件库或者HAL）
- ADC DMA 方式编程 （固件库或者HAL）

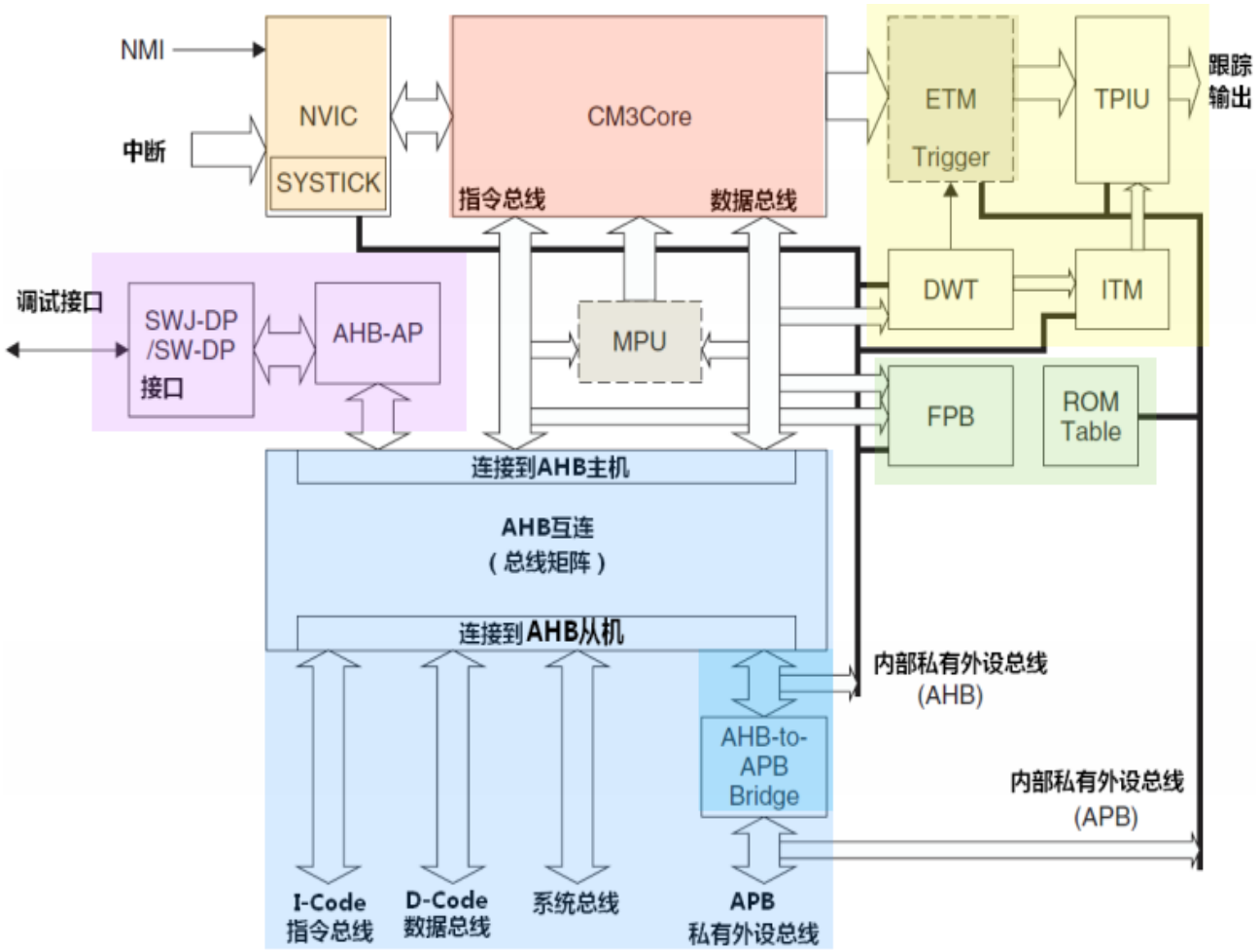
本章知识与能力要求

- 理解和掌握DMA的基本概念、适用场合;
- 理解STM32的DMA结构及工作原理;
- 掌握基于STM32标准外设库开发DMA应用程序的方法;
- 熟悉STM32标准外设库和HAL库中有关DMA的库函数;
- 了解基于STM32CubeMX进行DMA开发的方法。

STM32 DMA

1. DMA基础理论知识
2. STM32的DMA模块
3. DMA模块的标准外设库接口函数及应用
4. DMA模块的HAL库接口函数及应用

Cortex-M3内核

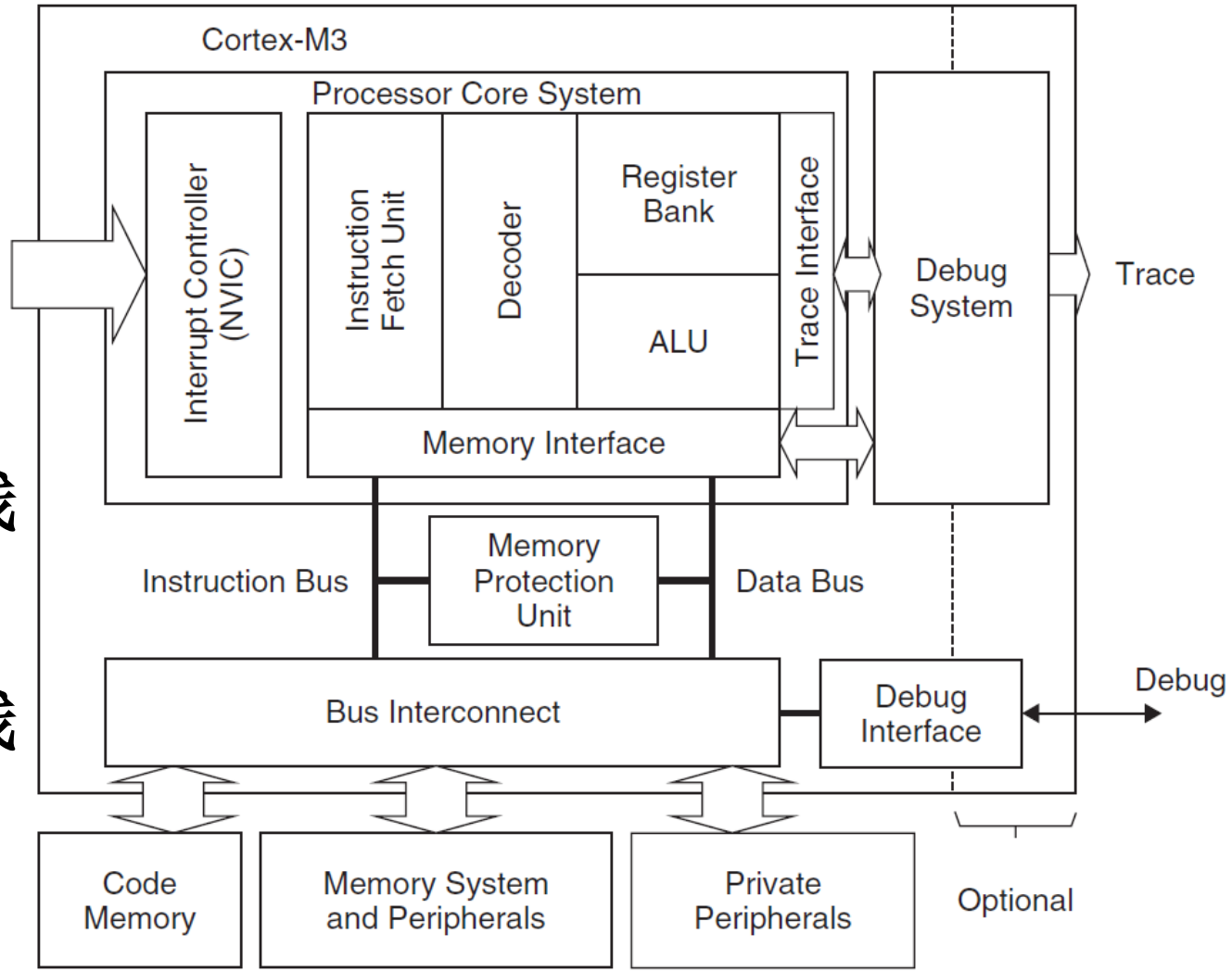


Cortex-M3内核架构框图

- **Cortex-M3 Core**：包括指令提取单元、译码单元、寄存器组和ALU等；
- **嵌套向量中断控制器（NVIC）**：用于实现快速、低延迟的异常中断处理；
- **系统时钟（SYSTICK）**：24位倒计时计数器，可作为系统定时器，用于产生定时中断；
- **存储器保护单元（MPU）**：用于对存储区域的访问保护，防止用户程序破坏存储区域的关键数据；
- **总线矩阵（AHB互连网络）**：用来将处理器内核和调试接口连接到不同功能的外部总线上，提供并行数据传输功能，还提供附加数据传送功能。

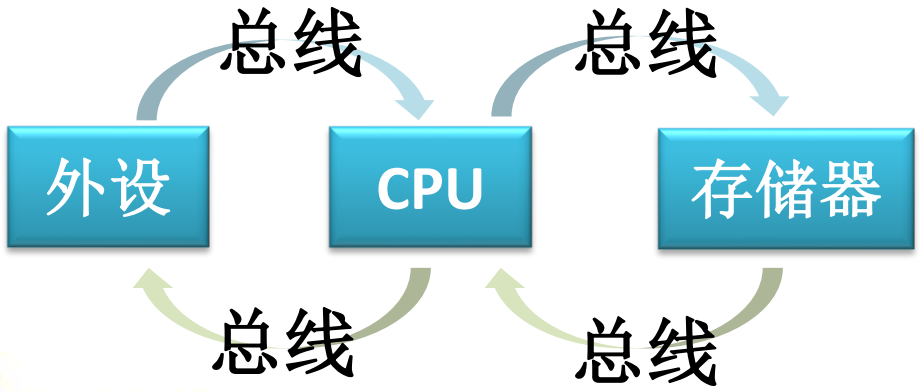
Cortex-M3内核

- Cortex-M3是基于ARMv7-M体系结构设计的32位的处理器内核
- 数据宽度、寄存器位数和存储器位数都是32位的
- 采用基于哈佛结构的三级流水线内核
- 拥有独立的数据总线和指令总线
- 数据总线和指令总线共享一个存储器空间，总寻址空间为4GB



1 DMA基础理论知识

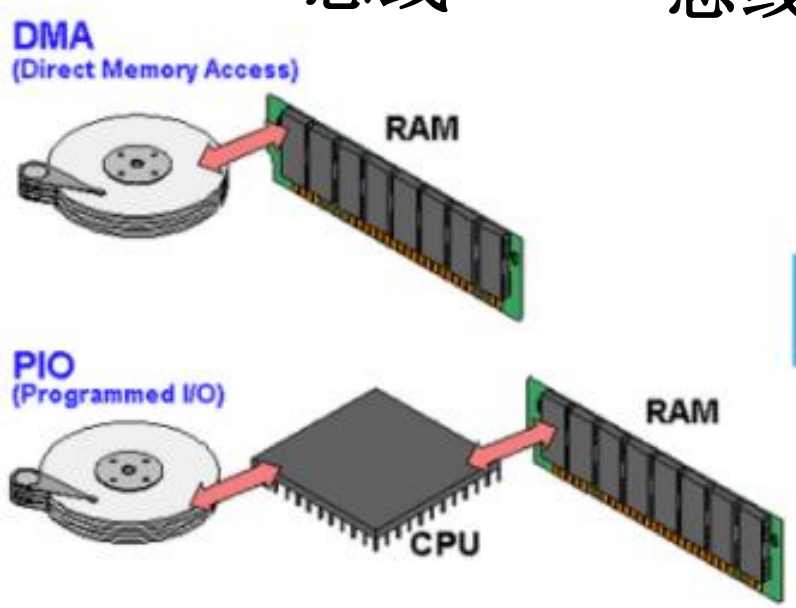
一般的数据处理流程:
在外设与存储器传输数据过程中，CPU全程参与



不需要处理数据时

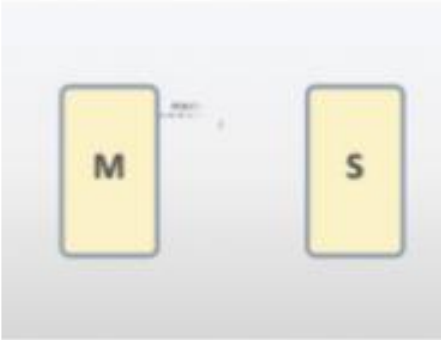
由于外设和存储器速度较低

导致CPU大量的时间和资源被浪费，造成CPU效率低下。



SD卡读写

屏幕显示



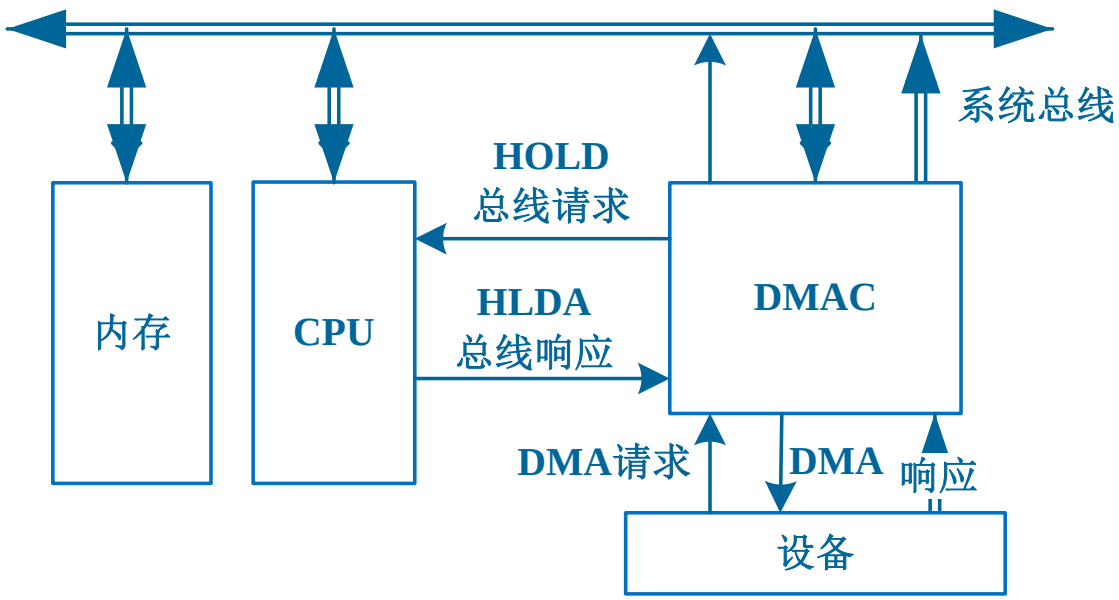
大量数据传输

1 DMA基础理论知识

直接存储器存取（简称DMA）

DMA传输方式是利用DMA控制器直接控制总线，在外设与存储器之间建立一条直接通道，不需要CPU的中转。

涉及到总线控制权转移问题，即DMA传输前，CPU需要把总线控制权交给DMA控制器，DMA传输结束后，DMA控制器再把总线控制权交还给CPU。



1 DMA基础理论知识

DMA传输过程

DMA请求

- 对DMA进行初始化，I/O设备准备好后，向DMA控制器发出DMA请求。

DMA响应

- DMA控制器对DMA请求进行优先级判别，向CPU发出总线请求信号。CPU响应总线请求，当CPU执行完当前总线周期就将总线控制权交给DMA控制器，完成DMA响应。

DMA传输

- DMA控制器获得总线控制权后，外设经硬件信号请求DMA传输或通过软件启动DMA请求，按配置好的相关参数，DMA控制器发出读写命令，开始进行数据传送。

DMA结束

- DMA控制器对传输的数据进行计数，判断数据是否传输完成，当传输完成，DMA控制器将总线控制权交还给CPU，并向I/O设备发出结束信号。

2 STM32的DMA模块

2.1 DMA内部结构

2.2 DMA优先权

2.3 DMA中断请求

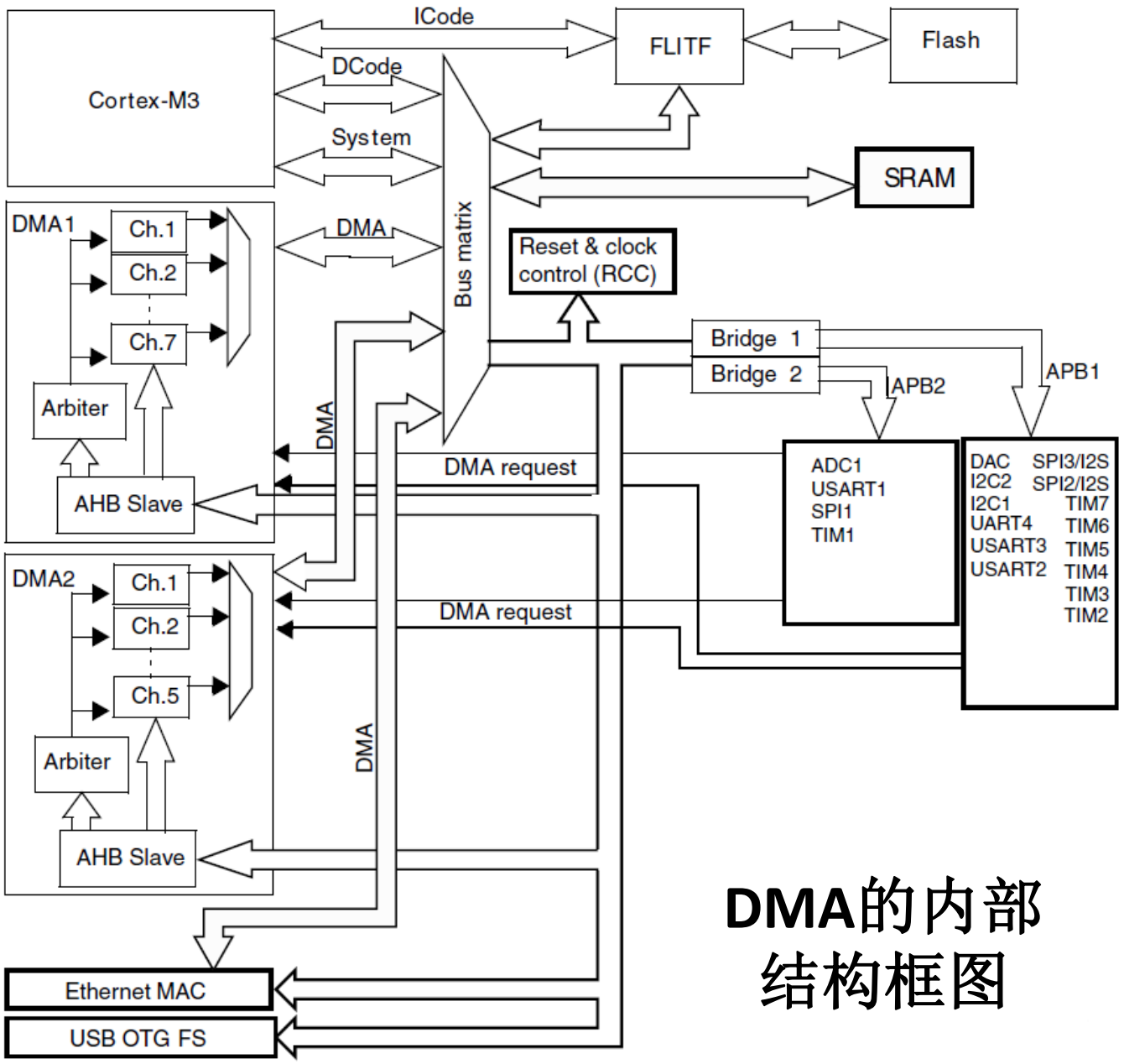
2.1 DMA内部结构

STM32最多有2个DMA控制器：

- **DMA1有7个通道**
- **DMA2有5个通道**

每一个通道专门用来管理一个或多个外设对存储器访问的请求。

中小容量的**STM32F103**系列 只有**DMA1**，**STM32F103RCT ~ ZET6**大容量芯片，有**DMA1**和**DMA2**两个**DMA**控制器。



DMA的内部结构框图

2.1 DMA内部结构

STM32最多有2个DMA控制器：

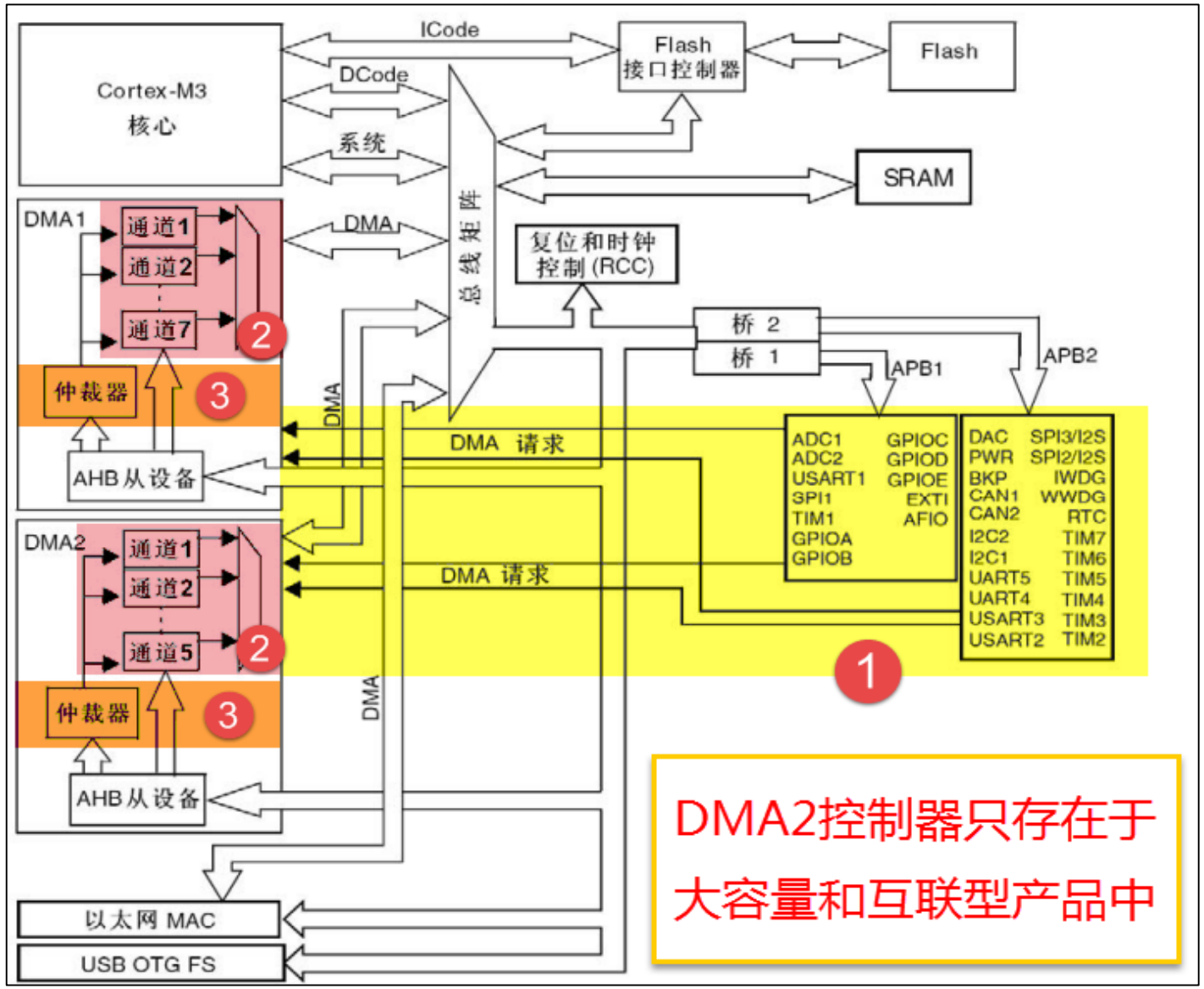
- **DMA1有7个通道**
- **DMA2有5个通道**

中小容量的STM32F103系列只有DMA1，大容量芯片有DMA1和DMA2两个DMA控制器。

1-DMA请求

2-通道

3-仲裁器



DMA2控制器只存在于大容量和互联型产品中

DMA的内部结构框图

2.1 DMA内部结构

DMA1各个通道的DMA请求

外设 (TIMx (x=1,2,3,4)、ADC1、SPI1、IICx (x=1,2)、USARTx (x=1,2,3)) 产生的DMA1请求，传送到DMA1控制器，同一时刻只能有一个请求有效。

外设	通道1	通道2	通道3	通道4	通道5	通道6	通道7
ADC1	ADC1						
SPI/I2S		SPI1_RX	SPI1_TX	SPI/I2S_RX	SPI/I2S_TX		
USART		USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
IIC				IIC2_TX	IIC2_RX	IIC1_TX	IIC1_RX
TIM1		TIM1_CH1	TIM1_CH2	TIM1_TX4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
TIM2	TIM2_CH3	TIM2_UP			TIM2_CH1		TIM2_CH2 TIM2_CH4
TIM3		TIM3_CH3	TIM3_CH4 TIM3_UP			TIM3_CH1 TIM3_TRIG	
TIM4	TIM4_CH1			TIM4_CH2	TIM4_CH3		TIM4_UP

2.1 DMA内部结构

DMA2各个通道的DMA请求

外设 (TIMx (x=5,6,7,8)、ADC3、SPI/I2S3、USART4、DAC通道1、2和SDIO) 产生5个通道的DMA2请求传送到DMA2控制器，同一时刻只能有一个请求有效。

外设	通道1	通道2	通道3	通道4	通道5
ADC3					ADC3
SPI/I2S3	SPI/I2S3_RX	SPI/I2S3_TX			
USART4			USART4_RX		USART4_TX
SDIO				SDIO	
TIM5	TIM5_CH4 TIM5_TRIG	TIM5_CH3 TIM5_UP		TIM5_CH2	TIM5_CH1
TIM6/ DAC通道1			TIM6_UP/ DMA通道1		
TIM7/ DAC通道2				TIM7_UP/ DMA通道2	
TIM8	TIM8_CH3 TIM8_UP	TIM8_CH4 TIM8_TRIG TIM8_COM	TIM8_CH1		TIM8_CH2

2.2 DMA优先权

当有多个DMA请求时，DMA控制器通过内部的仲裁器进行优先权管理。

通道的优先权分为4个等级，高优先级的通道优先获得总线响应。

最高优先级（Very High）

高优先级（High）

中等优先级（Medium）

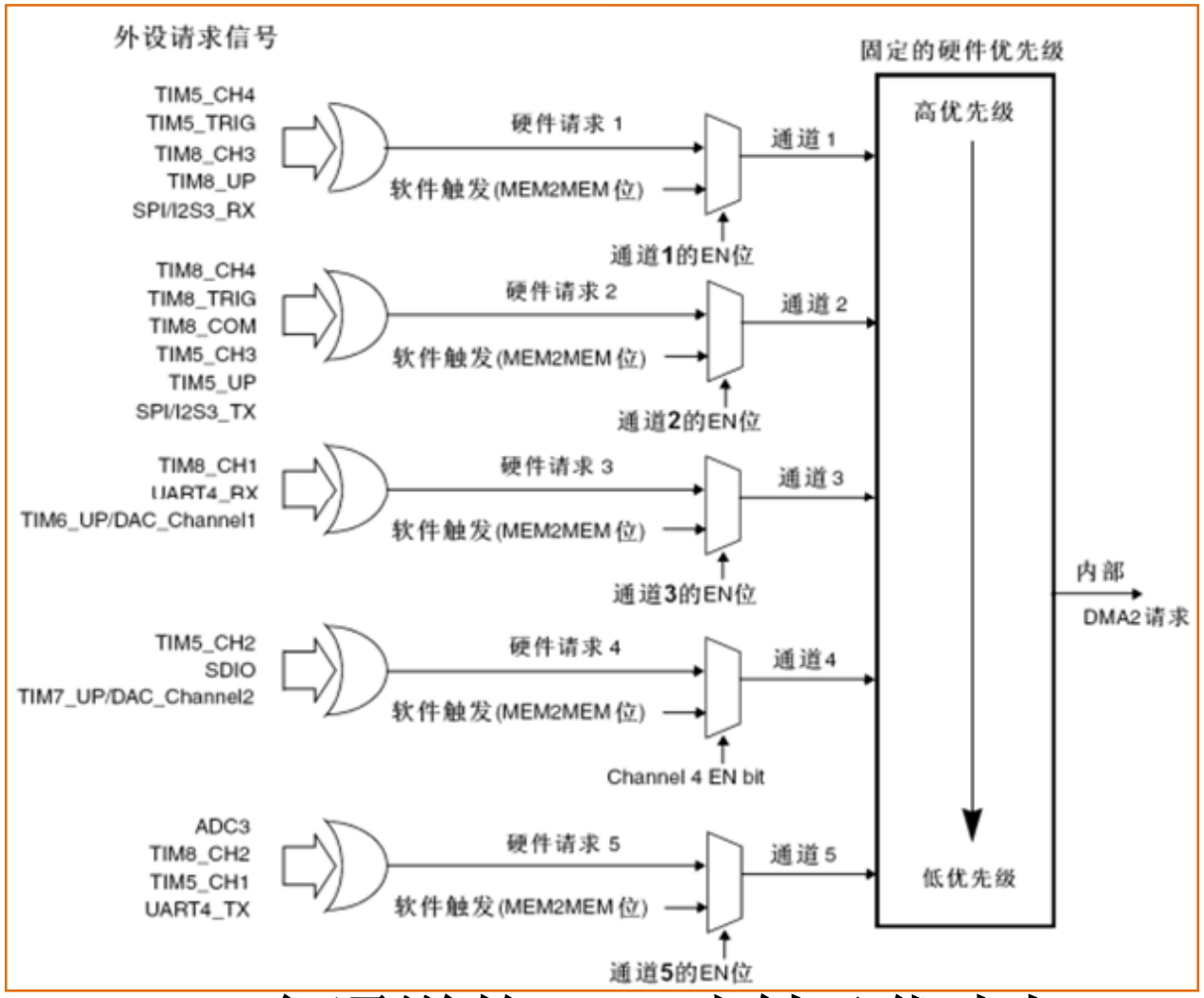
低优先级（Low）

如果2个请求有相同的软件优先级，则较低编号的通道比较高编号的通道有较高的优先权。例如，通道2优先于通道4。

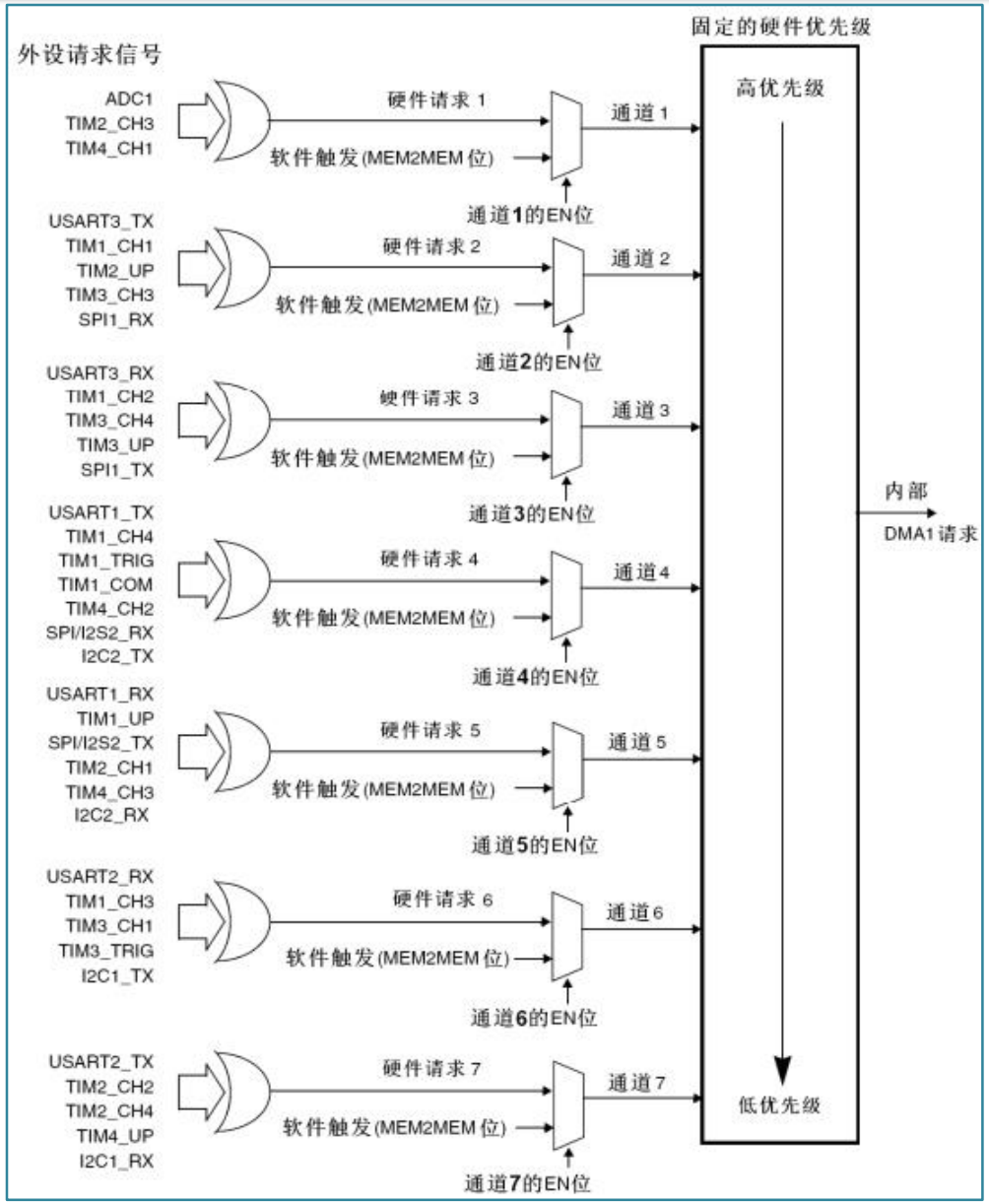
在大容量和互联型产品中，DMA1控制器拥有高于DMA2控制器的优先级。

1、软件阶段，DMA_CCRx：PL[1:0]。
2、硬件阶段，通道编号小的优先级大，DM1的优先级高于DMA2的优先级

2.2 DMA优先权



DMA2各通道的DMA映射及优先权



DMA1各通道的DMA映射及优先权

2.3 DMA中断请求

DMA的每个通道都可以在DMA传输过程中触发中断，可通过设置相应寄存器的不同位来打开这些中断。

DMA中断事件主要有：

- HT（Half Transfer，传输一半）
- TC（Transfer Complete，传输完成）
- TE（Transfer Error，传输错误）

分别对应三个中断标志：HTIF、TCIF、TEIF，每个中断标志都有允许控制位。

中断事件标志位	中断使能控制位	中断事件
HTIF	HTIE	传输过半
TCIF	TCIE	传输完成
TEIF	TEIE	传输错误

2.3 DMA使用要点

✓ DMA与外设建立连接

要使DMA与外设建立有效连接，这不是DMA自身的事情，是各个外设的事情，每个外设都有一个xxx_DMACmd(XXXx, Enable)函数，如果使DMA与ADC建立有效联系，就使用以下函数：

ADC_DMACmd(ADC1, Enable)

✓ DMA三种工作模式的选用

- 非循环模式 — DMA_Mode_Normal

也即正常模式，当一次DMA数据传输完后，停止DMA传送。需要重组设置传送数量和启动后，才能进行下一次DMA数据传输

- 循环模式 — DMA_Mode_Circular

用于处理循环缓冲区和连续的数据传输（如ADC的扫描模式）

在DMA_CCRx寄存器中的“CIRC”位可开启这一功能。当启动了循环模式，数据传输的数目变为0时，将会自动地被恢复成配置通道时设置的初值，DMA操作将会继续进行

- 存储器到存储器模式 — DMA_M2M_Enable

2.3 DMA使用要点

数据从哪里来，要到哪里去

```
uint32_t DMA_PeripheralBaseAddr;    // 外设地址
uint32_t DMA_MemoryBaseAddr;        // 存储器地址
uint32_t DMA_DIR;                   // 传输方向
```

1、外设地址，DMA_CPAR

2、存储器地址，DMA_CMAR

3、传输方向，DMA_CCR:DIR

什么时候传输结束

```
uint32_t DMA_Mode;                  // 模式选择
```

1、模式选择，DMA_CCRx:CIRC

2、传输过半，传输完成/出错，DMA_ISR

数据要传多少，传的单位是什么

```
uint32_t DMA_BufferSize;            // 传输数目
uint32_t DMA_PeripheralInc;         // 外设地址增量模式
uint32_t DMA_MemoryInc;             // 存储器地址增量模式
uint32_t DMA_PeripheralDataSize;    // 外设数据宽度
uint32_t DMA_MemoryDataSize;        // 存储器数据宽度
```

1、传输数目，DMA_CNDTR

2、外设地址是否递增，DMA_CCRx:PINC

3、存储器地址是否递增，DMA_CCRx:MINC

4、外设数据宽度， DMA_CCRx:PSIZE

5、存储器数据宽度， DMA_CCRx:MSIZE

3 DMA模块的标准外设库接口函数及应用

1. DMA标准外设库接口函数
2. DMA标准外设库的配置过程
3. DMA标准外设库应用实例

3.1 DMA标准外设库接口函数

DMA标准外设库支持 11种库函数

函数名称	功能描述
DMA_DeInit	将DMA的通道x寄存器重设为缺省值
DMA_Init	根据DMA_InitStruct中指定的参数初始化DMA的通道x寄存器
DMA_StrucInit	把DMA_InitStruct中的每个参数按缺省值填入
DMA_Cmd	使能或者失能指定的通道x
DMA_ITConfig	使能或者失能指定的通道x中断
DMA_GetCurrDataCounter	返回当前DMA通道x剩余的待传输数据个数
DMA_SetCurrDataCounter	设置DMA通道x进行DMA传输的当前数据个数
DMA_GetFlagStatus	检查指定的DMA通道x标志位设置与否
DMA_ClearFlag	清除DMA通道x待处理标志位
DMA_GetITStatus	检查指定的DMA通道x中断发生与否
DMA_ClearITPendingBit	清除DMA通道x中断待处理标志位

3.1 DMA标准外设库接口函数

在stm32f10x_dma.h头文件中
声明了DMA所有的库函数

stm32f10x_dma.c | stm32f10x_dma.h

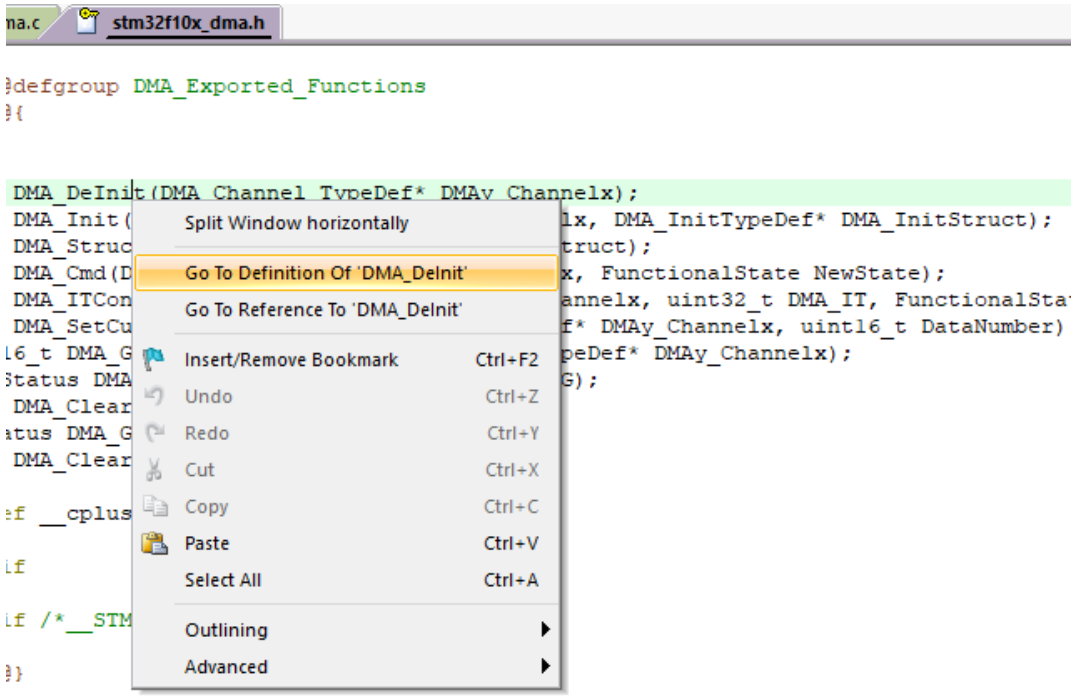
```
/** @defgroup DMA_Exported_Functions
  * @{
  */
void DMA_DeInit(DMA_Channel_TypeDef* DMAy_Channelx);
void DMA_Init(DMA_Channel_TypeDef* DMAy_Channelx, DMA_InitTypeDef* DMA_InitStruct);
void DMA_StructInit(DMA_InitTypeDef* DMA_InitStruct);
void DMA_Cmd(DMA_Channel_TypeDef* DMAy_Channelx, FunctionalState NewState);
void DMA_ITConfig(DMA_Channel_TypeDef* DMAy_Channelx, uint32_t DMA_IT, FunctionalState NewState);
void DMA_SetCurrDataCounter(DMA_Channel_TypeDef* DMAy_Channelx, uint16_t DataNumber);
uint16_t DMA_GetCurrDataCounter(DMA_Channel_TypeDef* DMAy_Channelx);
FlagStatus DMA_GetFlagStatus(uint32_t DMAy_FLAG);
void DMA_ClearFlag(uint32_t DMAy_FLAG);
ITStatus DMA_GetITStatus(uint32_t DMAy_IT);
void DMA_ClearITPendingBit(uint32_t DMAy_IT);

#ifdef __cplusplus
}
#endif

#endif /*__STM32F10x_DMA_H */
/**
  * @}
  */

/**
  * @}
  */
```

DMA库函数



如果想要查看具体的函数定义，可在keil 工程中将光标放在想要查看的函数上，然后单击右键选择“Go To Definition of…”，即可跳转到相应函数的函数体

3.1 DMA标准外设库接口函数

参数2：根据DMA_InitStruct结构体变量中指定的参数初始化DMA的通道x，DMA_InitStruct是指向DMA_InitTypeDef结构体的指针，包含DMA通道的配置参数

DMA的初始化函数为：

void DMA_Init(DMA_Channel_TypeDef* DMAy_Channelx, DMA_InitTypeDef* DMA_InitStruct)

初始化DMA结构体成员，

DMA_InitTypeDef定义在stm32f10x_dma.h文件中

```
typedef struct
{
   uint32_t DMA_PeripheralBaseAddr;        // 外设基地址
   uint32_t DMA_MemoryBaseAddr;            // 存储器基地址
   uint32_t DMA_DIR;                       // 传输方向：外设是源还是目的
   uint32_t DMA_BufferSize;                // 缓冲区大小
   uint32_t DMA_PeripheralInc;             // 外设地址是否递增
   uint32_t DMA_MemoryInc;                 // 存储器地址是否递增
   uint32_t DMA_PeripheralDataSize;        // 外设数据宽度
   uint32_t DMA_MemoryDataSize;            // 存储器数据宽度
   uint32_t DMA_Mode;                      // DMA工作模式
   uint32_t DMA_Priority;                  // DMA通道优先权
   uint32_t DMA_M2M;                       // 是否存储器到存储器传输
} DMA_InitTypeDef;
```

3.1 DMA标准外设库接口函数

DMA_InitTypeDef结构体成员及其取值

DMA_InitTypeDef 结构体成员	取值范围
DMA_PeripheralBaseAddr（外设基地址）	直接给出外设的地址
DMA_MemoryBaseAddr（存储器基地址）	通常为用户程序定义的主存缓冲区首地址
DMA_DIR（传输方向）	DMA_DIR_PeripheralSRC，外设作为数据传输的来源 DMA_DIR_PeripheralDST，外设作为数据传输的目的地
DMA_BufferSize（缓冲区大小）	设置一次传输数据量的大小，可设为 0~65536
DMA_PeripheralInc（外设地址增量）	DMA_PeripheralInc_Enable，外设地址递增允许 DMA_PeripheralInc_Disable，外设地址不变
DMA_MemoryInc（存储器地址增量）	DMA_MemoryInc_Enable，内存地址递增 DMA_MemoryInc_Disable，内存地址不变
DMA_PeripheralDataSize（外设数据宽度）	DMA_PeripheralDataSize_Byte，外设数据宽度为8位 DMA_PeripheralDataSize_HalfWord，外设数据宽度为16位 DMA_PeripheralDataSize_Word，外设数据宽度为32位
DMA_MemoryDataSize（存储器数据宽度）	DMA_MemoryDataSize_Byte，存储器数据宽度为8位 DMA_MemoryDataSize_HalfWord，存储器数据宽度为16位 DMA_MemoryDataSize_Word，存储器数据宽度为32位
DMA_Mode（DMA工作模式）	DMA_Mode_Circular，工作在循环缓冲模式 DMA_Mode_Normal，工作在正常缓冲模式
DMA_Priority（DMA优先权）	DMA_Priority_VeryHigh，最高优先级；DMA_Priority_High，高优先级； DMA_Priority_Medium，中等优先级；DMA_Priority_Low，低优先级
DMA_M2M（存储器到存储器传输）	DMA_M2M_Enable，允许内存到内存的传输； DMA_M2M_Disable，禁止内存到内存的传输

3.2 DMA标准外设库的配置过程

库函数的配置过程（以DMA1通道4为例）

Step 1

使能DMA时钟

DMA挂接在AHB，所以使用RCC_AHBPeriphClockCmd()函数使能DMA时钟：

```
RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1,ENABLE);
```

Step 2

初始化DMA通道4参数

```
DMA_Init(DMA2_Channel4,
&DMA_InitStructure)
```

Step 3

允许DMA中断，如果使用中断，则要进行DMA中断配置

```
DMA_ITConfig(DMA1_Channel4,
DMA_IT_TC,ENABLE);
```

Step 4

使能DMA1通道4，启动传输

```
DMA_Cmd(DMA1_Channel4, ENABLE);
```

3.2 DMA标准外设库的配置过程

初始化DMA通道4参数

```
DMA_InitTypeDef DMA_InitStructure;
DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)(&(USART1->DR));/*设置DMA源数据寄存器地址*/
DMA_InitStructure.DMA_MemoryBaseAddr = (uint32_t)SendBuff; /*内存地址，要传输的变量的指针*/
DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralDST; /*方向：从内存到外设*/
DMA_InitStructure.DMA_BufferSize = SENDBUFF_SIZE;    /*传输大小DMA_BufferSize=SENDBUFF_SIZE*/
DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;          /*外设地址设置为不增*/
DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;                   /*内存地址设置为自增*/
DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Byte;   /*设置外设数据单位*/
DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Byte;           /*设置内存数据单位8bit*/
DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;                  /*设置DMA模式为不断循环模式*/
DMA_InitStructure.DMA_Priority = DMA_Priority_Medium;                     /*设置DMA的优先级：中*/
DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;                     /*禁止内存到内存的传输*/
DMA_Init(DMA1_Channel4, &DMA_InitStructure);
```

3.3 DMA标准外设库应用实例

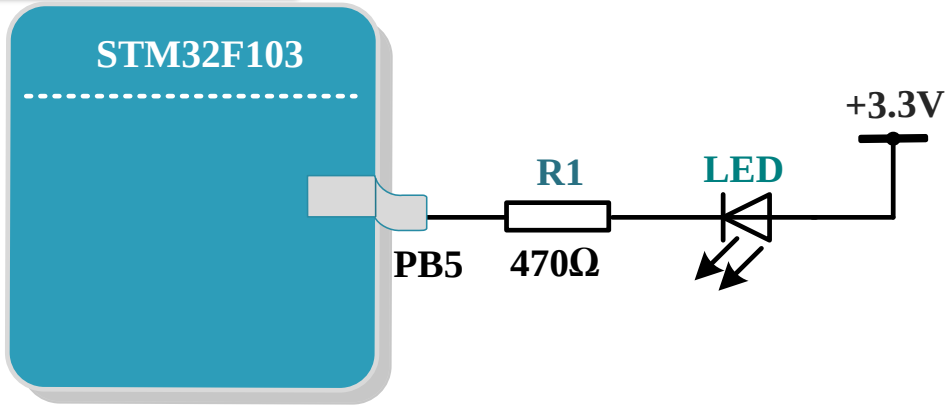
串口DMA方式传输数据

功能

通过DMA方式将数据发送到USART1接口，并通过串口调试助手显示结果。

为证明**DMA方式不需要CPU直接控制**，在主程序中通过循环点灯进行显示。

硬件设计



软件设计

- main.c 文件
- USARTx.c 文件
- USARTx.h 头文件

编译无错误后下载到开发板，通过串口调试助手可以观察到实验现象：**主程序LED灯一直循环闪烁，数据通过DMA方式不停地发送A到串口。**

3.3 DMA标准外设库应用实例

main.c文件

```
#include "stm32f10x.h"
#include "led.h"
#include "USARTx.h"
extern uint8_t SendBuff[SENDBUFF_SIZE];
// SENDBUFF_SIZE变量定义在USARTx.h头文件中

static void Delay(uint32_t time)    //不精确延时
{
  uint32_t i,j;
  for(i=0;i<time;++i)
  {
    for(j=0;j<10000;++j)
    {
      //空循环体，什么都不做， 纯延时
    }
  }
}
```

```
int main(void)
{   uint16_t i;
    LED_Init(); //LED灯初始化
    USARTx_Init_Config();      //USART1初始化
    USARTx_DMA_Config();    //DMA初始化
    printf("使用DMA方式传输串口数据\n");
      /*输入要发送的数据，作为DMA传输的数据源*/
    for(i=0;i<SENDBUFF_SIZE;i++)
    {    SendBuff[i] =  'A';
    }
        /* USART1 向DMA发出传输请求*/
    USART_DMACmd(USART1, USART_DMAReq_Tx, ENABLE);
    while(1)
    {   GPIO_SetBits(GPIOB, GPIO_Pin_5);      //熄灭LED灯
        Delay(1000);
        GPIO_ResetBits(GPIOB, GPIO_Pin_5);  //点亮LED灯
        Delay(1000);
    }
}
```

3.3 DMA标准外设库应用实例

USARTx.c文件

```
#include "USARTx.h"
uint8_t SendBuff[SENDBUFF_SIZE];
void USARTx_Init_Config(void)
{
   GPIO_InitTypeDef GPIO_InitStructure;
   USART_InitTypeDef USART_InitStructure;
   RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1|
 RCC_APB2Periph_GPIOA, ENABLE);
   USART_DeInit(USART1);
      // USART1_TX   PA.9
   GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
   GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
   GPIO_Init(GPIOA, &GPIO_InitStructure);
      // USART1_RX  PA.10
   GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
   GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_IN_FLOATING;
   GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```
    //USART1 参数配置
USART_InitStructure.USART_BaudRate = 115200;
USART_InitStructure.USART_WordLength =
USART_WordLength_8b;
USART_InitStructure.USART_StopBits =
USART_StopBits_1;
USART_InitStructure.USART_Parity = USART_Parity_No;
USART_InitStructure.USART_HardwareFlowControl =
USART_HardwareFlowControl_None;
USART_InitStructure.USART_Mode
   = USART_Mode_Rx | USART_Mode_Tx;
USART_Init(USART1, &USART_InitStructure);
USART_Cmd(USART1, ENABLE);
   /* 清除发送完成标志*/
USART_ClearFlag(USART1,
USART_FLAG_TC|USART_FLAG_TXE|USART_FLAG_RXNE);
}
```

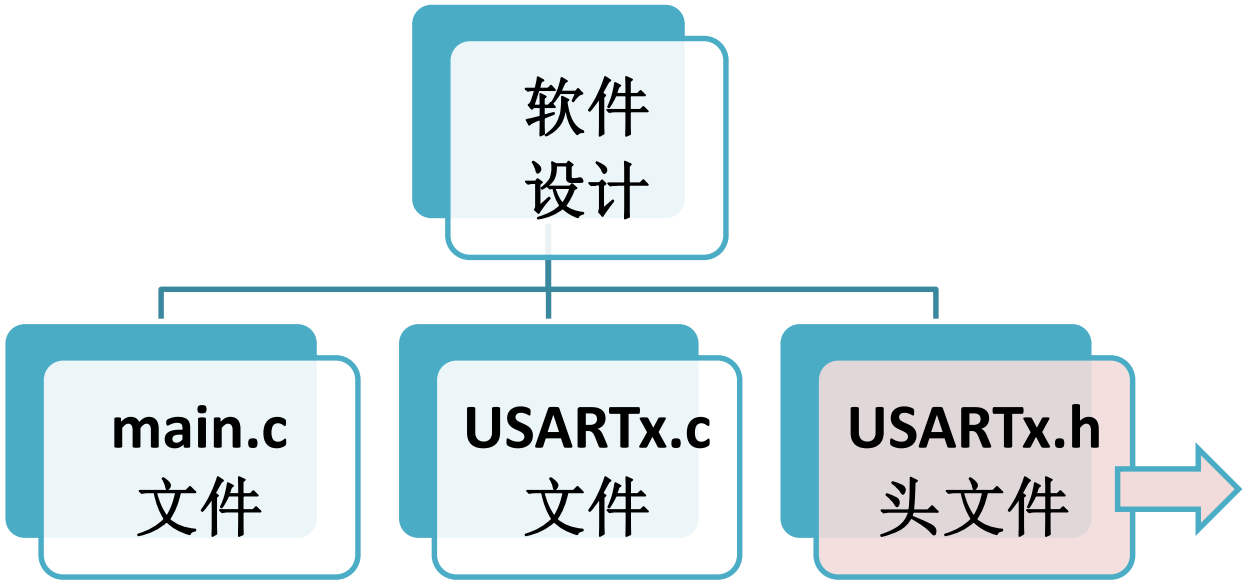
3.3 DMA标准外设库应用实例

USARTx.c文件

```
void USARTx_DMA_Config(void)
{
  DMA_InitTypeDef DMA_InitStructure;
   /*开启DMA时钟*/
  RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1,
ENABLE);
   /*设置DMA源：串口数据寄存器地址*/
  DMA_InitStructure.DMA_PeripheralBaseAddr
=(uint32_t)( &(USART1->DR) );
   /*内存地址，要传输的变量的指针*/
  DMA_InitStructure.DMA_MemoryBaseAddr =
(uint32_t)SendBuff;
   /*方向：从内存到外设*/
  DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralDST;
   /*传输大小DMA_BufferSize=SENDBUFF_SIZE*
  DMA_InitStructure.DMA_BufferSize = SENDBUFF_SIZE;
   /*外设地址设置为不增*/
  DMA_InitStructure.DMA_PeripheralInc =
DMA_PeripheralInc_Disable;
```

```
   /*内存地址设置为自增*/
  DMA_InitStructure.DMA_MemoryInc =
DMA_MemoryInc_Enable;
   /*设置外设数据单位*/
  DMA_InitStructure.DMA_PeripheralDataSize =
DMA_PeripheralDataSize_Byte;
   /*设置内存数据单位8bit*/
  DMA_InitStructure.DMA_MemoryDataSize =
DMA_MemoryDataSize_Byte;
   /*设置DMA模式为不断循环模式*/
  DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
   /*设置DMA的优先级：中*/
  DMA_InitStructure.DMA_Priority = DMA_Priority_Medium;
   /*禁止内存到内存的传输*/
  DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
   /*配置DMA1的通道4*/
  DMA_Init(DMA1_Channel4, &DMA_InitStructure);
   /*使能DMA*/
  DMA_Cmd (DMA1_Channel4, ENABLE);
   /*设置DMA发送完成后产生中断 */
  DMA_ITConfig (DMA1_Channel4, DMA_IT_TC, ENABLE);
}
```

3.3 DMA标准外设库应用实例



USARTx.h文件

```
#ifndef __USARTx_H
#define __USARTx_H

#include "stm32f10x.h"
#include <stdio.h>

#define  SENDBUFF_SIZE    100

void USARTx_Init_Config(void);
void USARTx_DMA_Config(void);

int fputc(int ch, FILE *f);
int fgetc(FILE *f);

#endif
```

3.4 DMA标准外设库应用实例

ADC DMA 实例

```
ADCx_GPIO_Config();
        // Mode

ADCx_Mode_Config();
        // ADC clk
        // DMA mode
```

扩展：引入串口DMA方式传输数据

DAC + DMA 输出正弦波实例

```
DAC_Config();
        // Tim TRGO
DAC_TIM_Config();
        // OutputTrig
DAC_DMA_Config();
        // BaseAddr
```

4.1 DMA的HAL库接口函数

DMA模块的HAL库相关的接口函数定义在stm32f1xx_hal_dma.c源文件中，在stm32f1xx_hal_dma.h头文件中可以查看相关函数的声明以及结构体定义。

类型	函数及功能描述
初始化及复位函数	HAL_DMA_Init(DMA_HandleTypeDef *hdma); 功能描述：DMA初始化函数
	HAL_DMA_DeInit (DMA_HandleTypeDef *hdma); 功能描述：DMA复位函数
引脚功能操作函数	HAL_DMA_Start (DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength); 功能描述：启动DMA传输 (DMA对象, 源数据地址, 目的地址, 数据长度)
	HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength); 功能描述：DMA中断开始
	void HAL_DMA_IRQHandler(DMA_HandleTypeDef *hdma); 功能描述：DMA中断处理函数
外设状态函数	HAL_DMA_GetState(DMA_HandleTypeDef *hdma); 功能描述：获取DMA状态函数
	HAL_DMA_GetError(DMA_HandleTypeDef *hdma); 功能描述：获取DMA错误函数

小结

DMA基础

理论知识

STM32的DMA模块

- DMA内部结构
- DMA优先权
- DMA中断请求

DMA模块的
标准外设库接口函数及应用

- 接口函数
- 配置过程
- 应用实例

DMA模块HAL库
接口函数及应用

- 接口函数
- 应用实例

AI 小结

Ask AI

STM32

The End!

STM32 HAL库与标准库的区别

一、配置寄存器在51中比较常见，通过汇编语言直接操作寄存器实现功能；这种方法到了STM32就变得不太容易行得通。因为STM32的寄存器数量是51的数十倍，如此多的寄存器根本无法全部记忆，开发时需要经常翻查芯片数据手册，此时直接操作寄存器就非常费力。但还是会有一部分人喜欢去直接操作寄存器，这样更接近原理，知其然也知其所以然。

二、标准库应运而生：ST公司为每款芯片都编写了一份库文件。在这些.c / .h文件中，包括一些常用量的宏定义，把一些外设也通过结构体变量封装起来，如GPIO时钟等。所以我们只需要配置结构体变量成员就可以修改外设的配置寄存器，从而选择不同的功能。也是目前最多人使用的方式，也是学习STM32接触最多的一种开发方式。

三、HAL库是ST公司目前主推的开发方式。它的出现比标准库要晚，都是为了节省程序开发的时期，而且HAL库更高效；如果说标准库把实现功能需要配置的寄存器集成了，那么HAL库的一些函数甚至可以做到某些特定功能的集成。也就是说，同样的功能，标准库可能要用几句话，HAL库只需用一句话就够了。并且HAL库也很好的解决了程序移植的问题，不同型号的STM32芯片它的标准库是不一样的，而使用HAL库，只要使用的是相通的外设，程序基本可以完全复制粘贴。

四、综上，学习HAL库有一定的优势：模块化向来是世界的潮流，更方便的HAL库一定会迅速发展，低效的短板迟早会被硬件高度集成化所弥补。

当然啦，不能只学习HAL库，底层的原理必需是要懂的，这是每个学有所成的人都公认的事实，HAL库也不是万能的，结合对底层的理解相信一定会让你的开发水准大大提高。

STM32 HAL库与标准库的区别
- 浅谈句柄、MSP函数、Callback函数

1.句柄

句柄 (handle)，有多种意义，其中第一种是指程序设计，第二种是指Windows编程。现在大部分都是指程序设计/程序开发这类。

第一种解释：句柄是一种特殊的智能指针 。当一个应用程序要引用其他系统（如数据库、操作系统）所管理的内存块或对象时，就要使用句柄。

第二种解释：整个Windows编程的基础。一个句柄是指使用的一个唯一的整数值，即一个4字节(64位程序中为8字节)长的数值，来标识应用程序中的不同对象和同类中的不同的实例，诸如，一个窗口，按钮，图标，滚动条，输出设备，控件或者文件等。应用程序能够通过句柄访问相应的对象的信息，但是句柄不是指针，程序不能利用句柄来直接阅读文件中的信息。Windows大量使用了句柄来标识对象，是用来标志应用程序中建立的或是使用的唯一整数。

STM32的标准库中，句柄是一种**特殊的指针**，通常指向结构体！

在STM32的标准库中，句柄是一种特殊的指针，通常指向结构体！

在STM32的标准库中，假设我们要初始化一个外设（以USART为例），我们首先要初始化他们的各个寄存器。在标准库中，这些操作都是利用固件库结构体变量+固件库Init函数实现的：

```
USART_InitTypeDef   USART_InitStructure;
USART_InitStructure.USART_BaudRate = bound;                    //串口波特率
USART_InitStructure.USART_WordLength = USART_WordLength_8b;//字长为8位数据格式
USART_InitStructure.USART_StopBits = USART_StopBits_1; //一个停止位
USART_InitStructure.USART_Parity = USART_Parity_No;      //无奇偶校验位
USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
                                                          //无硬件数据流控制
USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模式
USART_Init(USART3, &USART_InitStructure);                  //初始化串口1
```

可以看到，要初始化一个串口，需要：

- 1、对六个位置进行赋值，
- 2、然后引用Init函数，

USART_InitStructure并不是一个全局结构体变量，而是只在函数内部的局部变量，初始化完成之后，USART_InitStructure就失去了作用。

```
typedef struct
{
  USART_TypeDef *Instance;   /*!< UART registers base address */
  UART_InitTypeDef Init;     /*!< UART communication parameters */
  uint8_t *pTxBuffPtr;       /*!< Pointer to UART Tx transfer Buffer */
  uint16_t TxXferSize;       /*!< UART Tx Transfer size */
  uint16_t TxXferCount;      /*!< UART Tx Transfer Counter */
  uint8_t *pRxBuffPtr;       /*!< Pointer to UART Rx transfer Buffer */
  uint16_t RxXferSize;       /*!< UART Rx Transfer size */
  uint16_t RxXferCount;      /*!< UART Rx Transfer Counter */
  DMA_HandleTypeDef *hdmatx;  /*!< UART Tx DMA Handle parameters */
  DMA_HandleTypeDef *hdmarx;  /*!< UART Rx DMA Handle parameters */
  HAL_LockTypeDef Lock;       /*!< Locking object */
  __IO HAL_UART_StateTypeDef State;  /*!< UART communication state */
  __IO uint32_t ErrorCode;           /*!< UART Error code */
} UART_HandleTypeDef;
```

2.MSP函数

MSP：MCU Specific Package 单片机的具体方案； MSP是指和MCU相关的初始化：
我们要初始化一个串口，首先要设置和 MCU 无关的东西，例如波特率，奇偶校验，停止位等，这些参数设置和 MCU 没有任何关系，可以使用 STM32F1，也可以是 STM32F2/F3/F4/F7上的串口。
而一个串口设备它需要一个 MCU 来承载，例如用 STM32F4 来做承载，PA9 做为发送，PA10 做为接收，MSP 就是要初始化 STM32F4 的 PA9, PA10，配置这两个引脚。
所以 HAL驱动方式的初始化流程就是：

HAL_USART_Init() —> HAL_USART_MspInit()

先初始化与 MCU无关的串口协议，再初始化与 MCU 相关的串口引脚。
在 STM32 的 HAL 驱动中HAL_PPP_MspInit()作为回调，被 HAL_PPP_Init()函数所调用。当我们需要移植程序到 STM32F1平台的时候，我们只需要修改 HAL_PPP_MspInit 函数内容而不需要修改 HAL_PPP_Init 入口参数内容。
在HAL库中，几乎每初始化一个外设就需要设置该外设与单片机之间的联系，比如IO口是否复用等等，可见，HAL库相对于标准库多了MSP函数之后，加上又可以配合句柄，移植性非常强，但与此同时却增加了代码量和代码的嵌套层级。
可以说各有利弊。

3. Callback函数

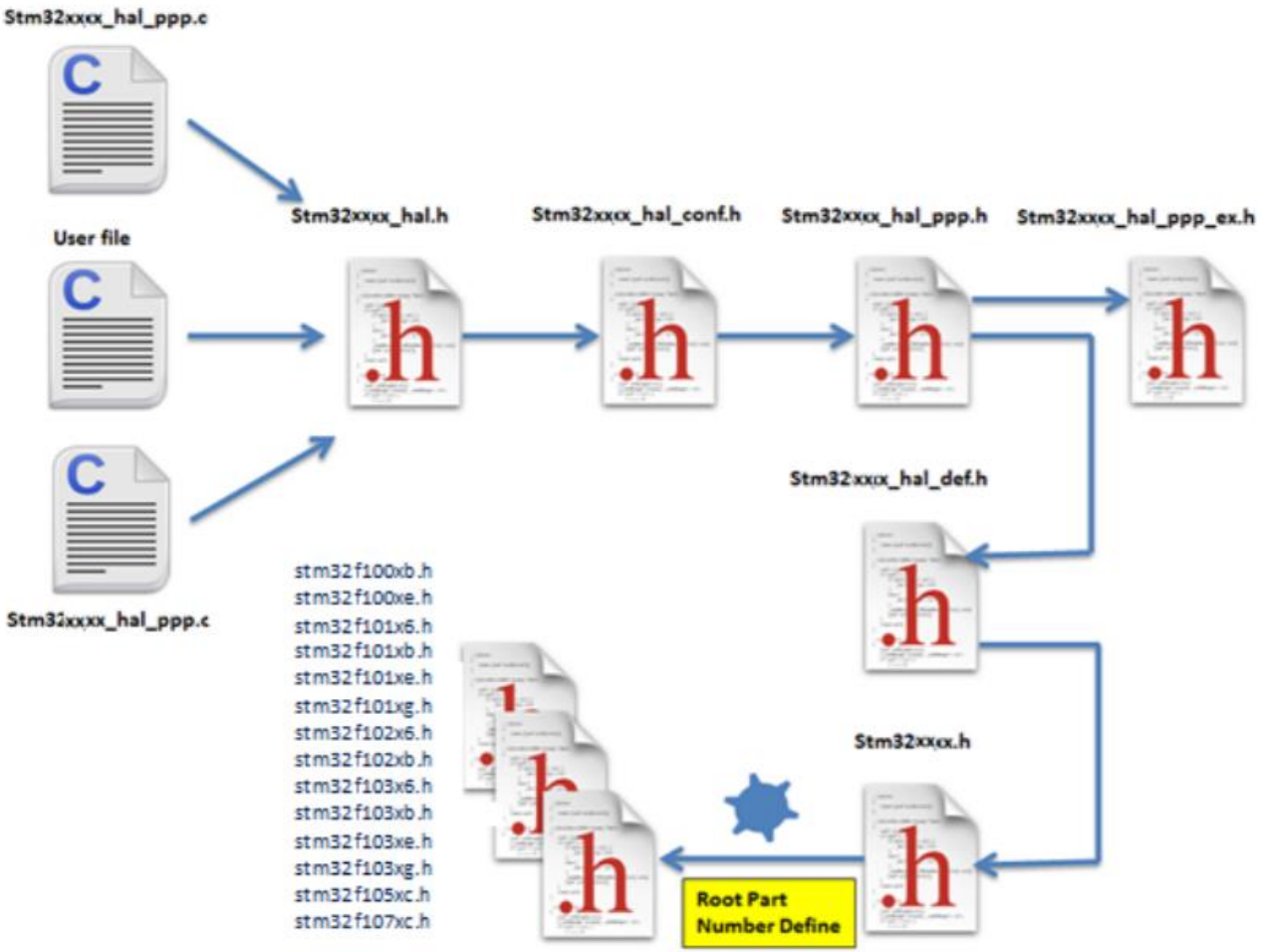
类似于MSP函数，个人认为Callback函数主要帮助用户应用层的代码编写。还是以USART为例，在标准库中，串口中断了以后，我们要先在中断中判断是否是接收中断，然后读出数据，顺便清除中断标志位，然后再是对数据的处理，这样如果我们在一个中断函数中写这么多代码，就会显得很混乱：

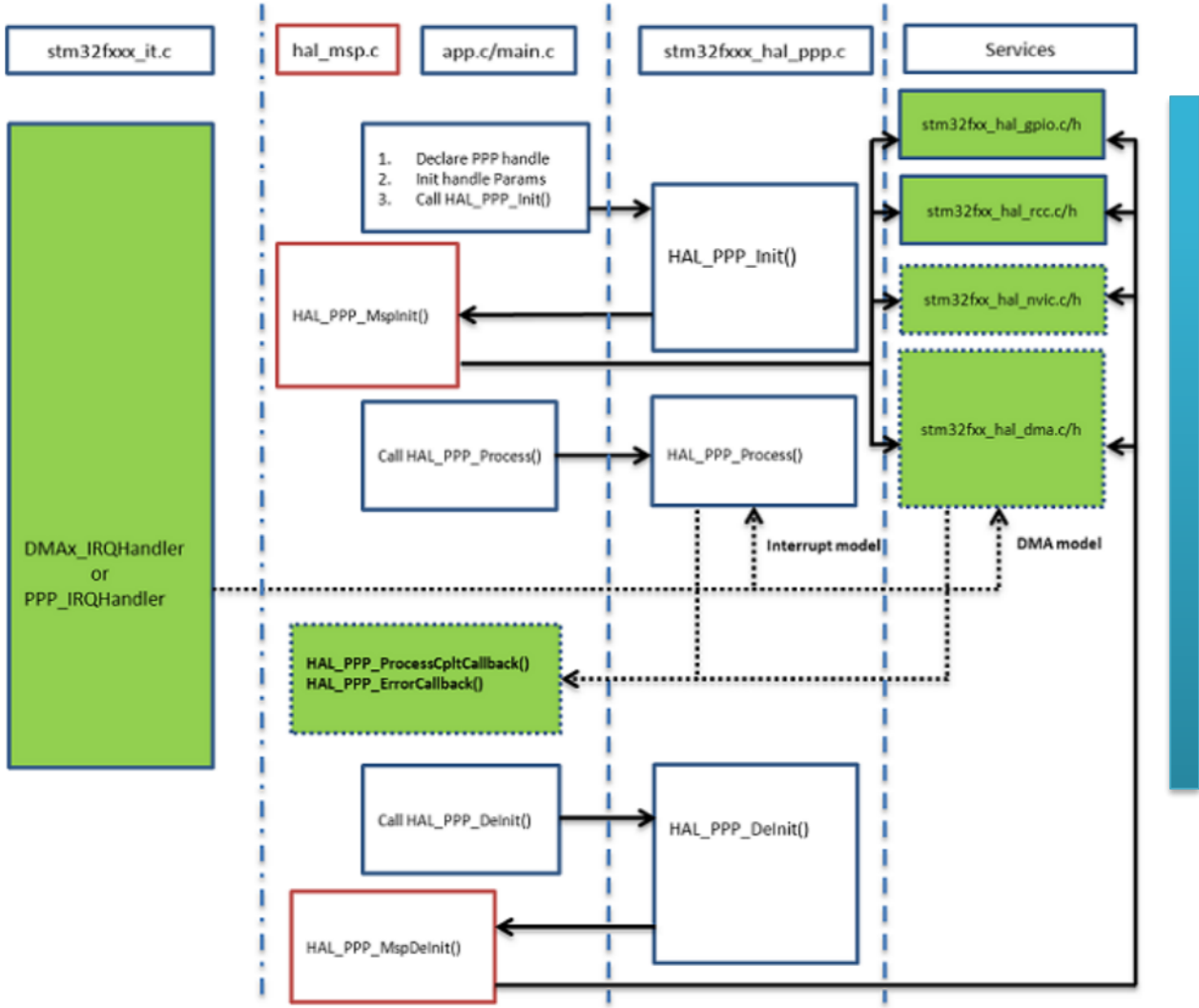
```
void USART3_IRQHandler(void)                    //串口1中断服务程序
{       u8 Res;
        if(USART_GetITStatus(USART3, USART_IT_RXNE) != RESET)
            //接收中断(接收到的数据必须是0x0d  0x0a结尾)
            {           Res =USART_ReceiveData(USART3);     //读取接收到的数据
            /*数据处理代码区*/                }
}
```

而在HAL库中，进入串口中断后，直接由HAL库中断函数进行托管：

```
void USART1_IRQHandler(void)
{
            HAL_UART_IRQHandler(&UART1_Handler);        //调用HAL库中断处理公用函数
            /****************省略无关代码*****************/
}
```

HAL库 结构

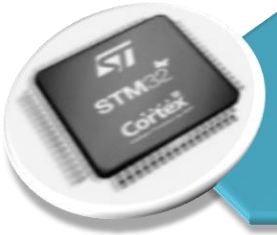




UM1850 User Manual

Description of STM32F1 HAL and low-layer drivers

4 DMA模块的HAL库接口函数及应用



4.1 DMA的HAL库接口函数

4.2 HAL库DMA应用实例

4.1 DMA的HAL库接口函数

DMA模块的HAL库相关的接口函数定义在stm32f1xx_hal_dma.c源文件中，在stm32f1xx_hal_dma.h头文件中可以查看相关函数的声明以及结构体定义。

STM32的HAL库常用DMA接口函数

类型	函数及功能描述
初始化及复位函数	HAL_DMA_Init(DMA_HandleTypeDef *hdma); 功能描述：DMA初始化函数
	HAL_DMA_DeInit (DMA_HandleTypeDef *hdma); 功能描述：DMA复位函数
引脚功能操作函数	HAL_DMA_Start (DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength); 功能描述：启动DMA传输
	HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength); 功能描述：DMA中断开始
	void HAL_DMA_IRQHandler(DMA_HandleTypeDef *hdma); 功能描述：DMA中断处理函数
外设状态函数	HAL_DMA_GetState(DMA_HandleTypeDef *hdma); 功能描述：获取DMA状态函数
	HAL_DMA_GetError(DMA_HandleTypeDef *hdma); 功能描述：获取DMA错误函数

4.1 DMA的HAL库接口函数

DMA启动传输函数
HAL_DMA_Start()

```
HAL_DMA_Start(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength);
```

四个参数

- hdma — 具体使用的DMA对象
- SrcAddress — DMA传输的源数据地址
- DstAddress — DMA传输的目的地址
- DataLength — DMA传输数据的长度

使用范例：

```
HAL_DMA_Start(huart->hdmatx, (u32)pData, (uint32_t)&huart->Instance->DR, 100);
    //开启DMA的串口传输
```

4.1 DMA的HAL库接口函数

DMA串口发送数据传输函数 HAL_UART_Transmit_DMA()

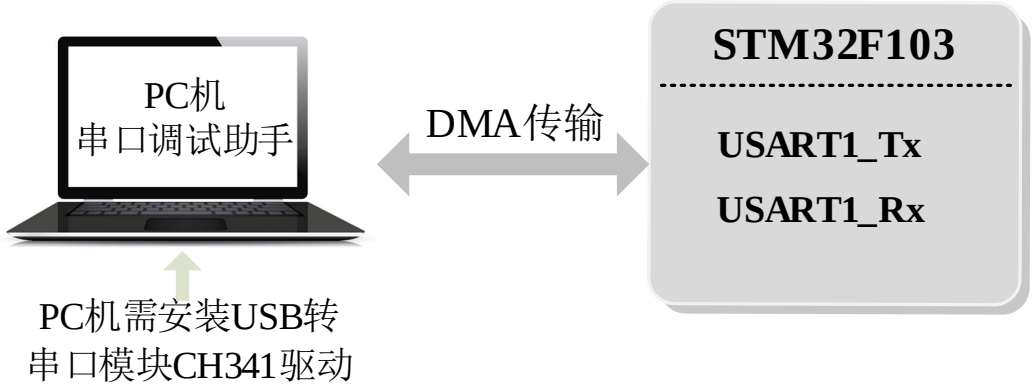
```
HAL_StatusTypeDef HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{
 uint32_t *tmp;
 if (huart->gState == HAL_UART_STATE_READY)
 {
  if ((pData == NULL) || (Size == 0U))
  {    return HAL_ERROR;   }
  __HAL_LOCK(huart);                          /*huart配置，上锁 */
  huart->pTxBuffPtr = pData;
  huart->TxXferSize = Size;
  huart->TxXferCount = Size;
  huart->ErrorCode = HAL_UART_ERROR_NONE;
  huart->gState = HAL_UART_STATE_BUSY_TX;
  huart->hdmatx->XferCpltCallback = UART_DMATransmitCplt;          /*设置UART的DMA传输完成时调用的回调函数*/
  huart->hdmatx->XferHalfCpltCallback = UART_DMATxHalfCplt;        /*设置UART的DMA传输完成一半时的回调函数 */
  huart->hdmatx->XferErrorCallback = UART_DMAError;                /*设置DMA传输错误的回调函数 */
  huart->hdmatx->XferAbortCallback = NULL;                         /*置DMA传输中止的回调函数*/
  tmp = (uint32_t *)&pData;                   /*设置存储器到外设的地址，开启中断，并使能UART的DMA传输的DMA通道*/
  HAL_DMA_Start_IT(huart->hdmatx, *(uint32_t *)tmp, (uint32_t)&huart->Instance->DR, Size);
  __HAL_UART_CLEAR_FLAG(huart, UART_FLAG_TC);                      /*清除串口TC中断标志位 */
  __HAL_UNLOCK(huart);                        /* huart解锁 */
  SET_BIT(huart->Instance->CR3, USART_CR3_DMAT);     /* 通过配置UART的CR3寄存器，使能DMA发送*/
  return HAL_OK;  }
 else {   return HAL_BUSY;   }
}
```

4.2 HAL库DMA应用实例

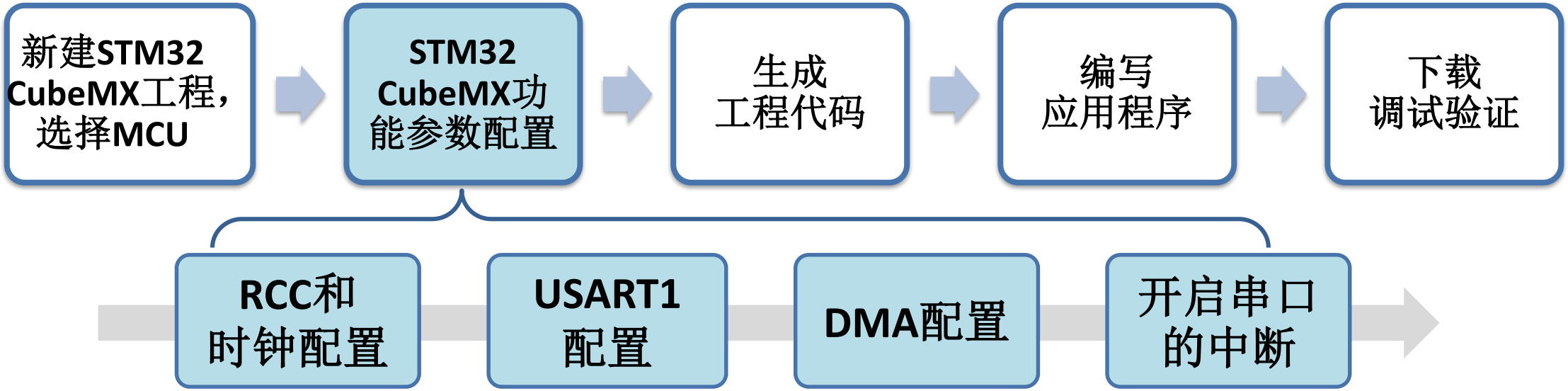
硬件设计

功能

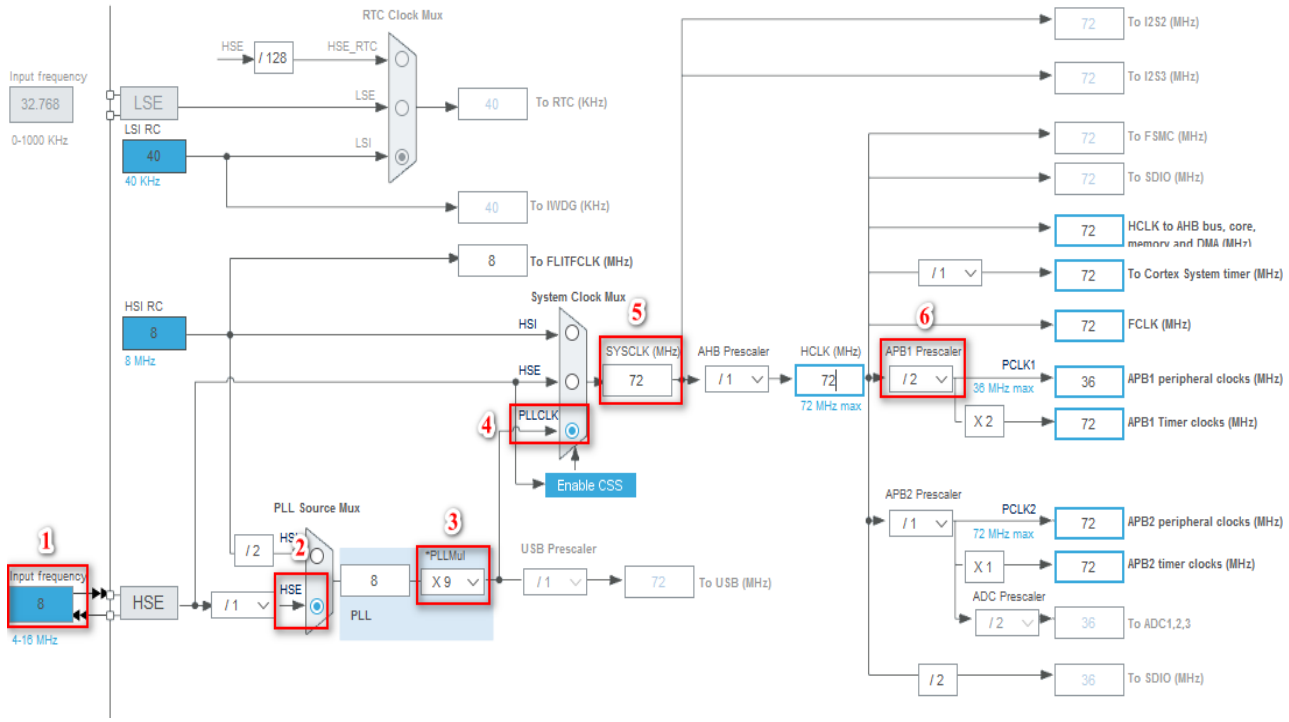
基于HAL库采用DMA方式实现USART串口1 收发数据，用于需要大批量数据的通信与信息交换；PC机的串口助手查看测试结果。



软件设计流程



软件设计——STM32CubeMX参数配置



新建一个STM32CubeMX工程，
根据自己的目标板选择相应的芯片

RCC配置: HSE选择“Crystal/Ceramic Resonator”(晶振/陶瓷谐振器)

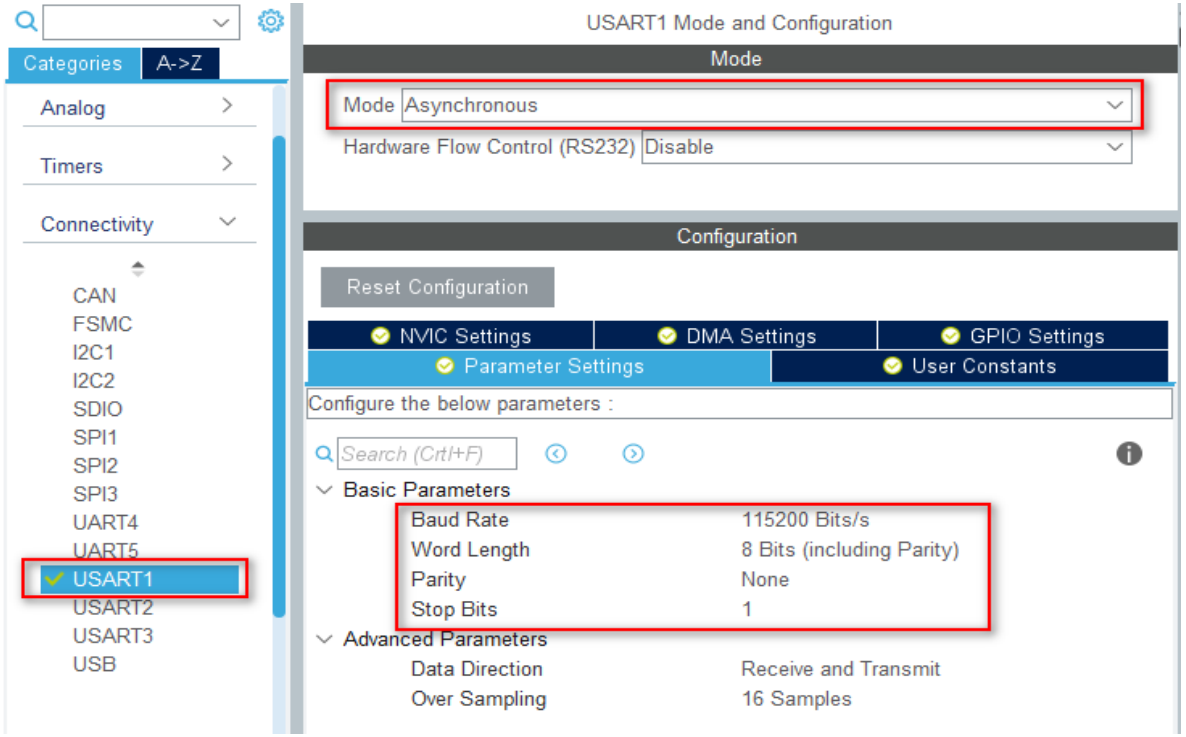
时钟配置

通过图形化方式直观地对系统时钟进行配置，系统时钟采用外部高速时钟，配置STM32F103系列芯片最大时钟为72MHz，配置APB2为72MHz，配置APB1为36MHz。

软件设计——STM32CubeMX功能参数配置

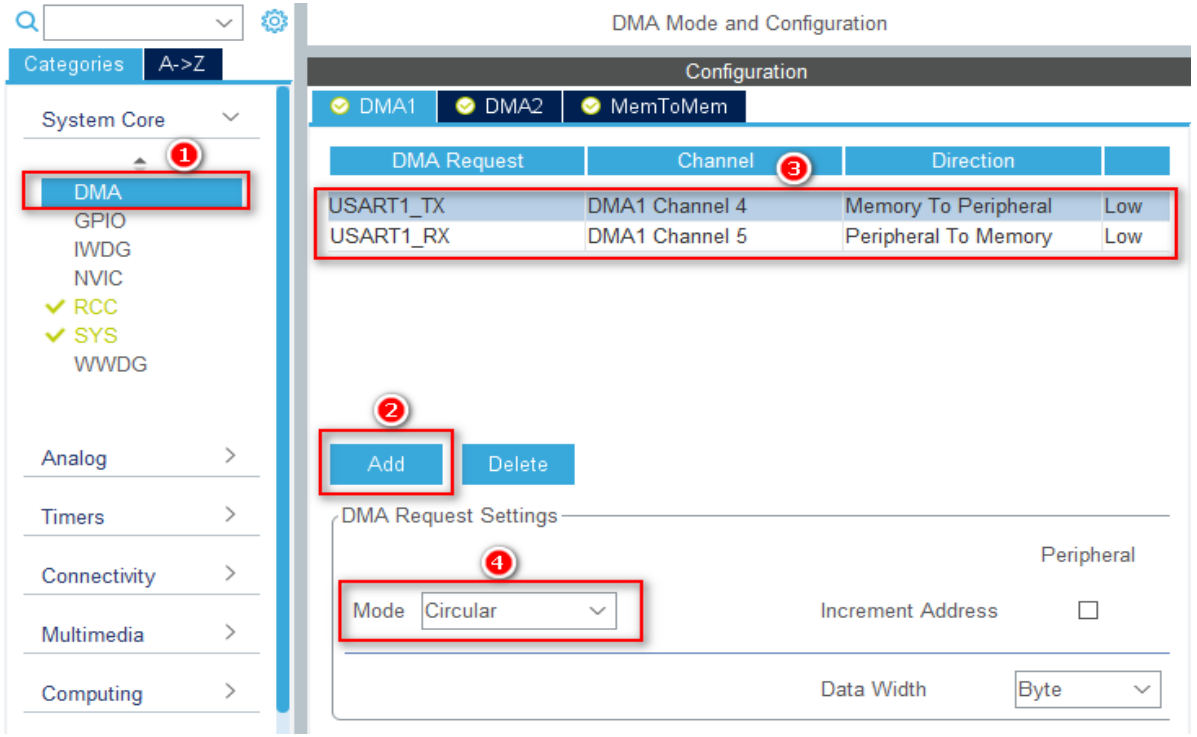
串口USART1配置

- 在"Connectivity"中选择"USART1"；
- 设置"MODE"为"Asynchronous"；
- 设置USART1的相关参数配置为默认的：115200、8、None和1



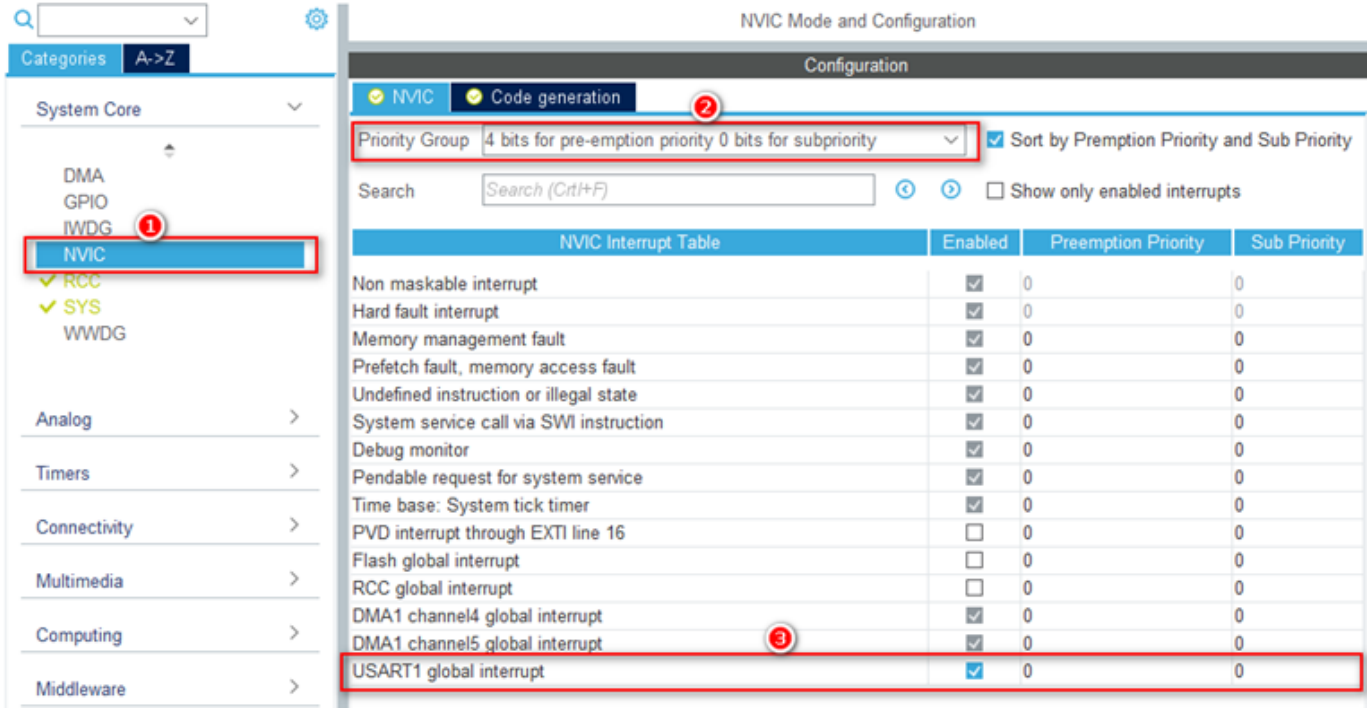
DMA配置

- 在"System Core"中"DMA"右侧的配置页面"Configuration"，单击"Add" 添加"USART1_TX"；
- "Mode"选择为"Circular"；
- "Channel"和"Direction"会自动选择；
- "Data Width"用默认的"Byte"；
- 按同样的操作方式添加"USART1_RX"



软件设计——STM32CubeMX功能参数配置

开启串口USART1的中断

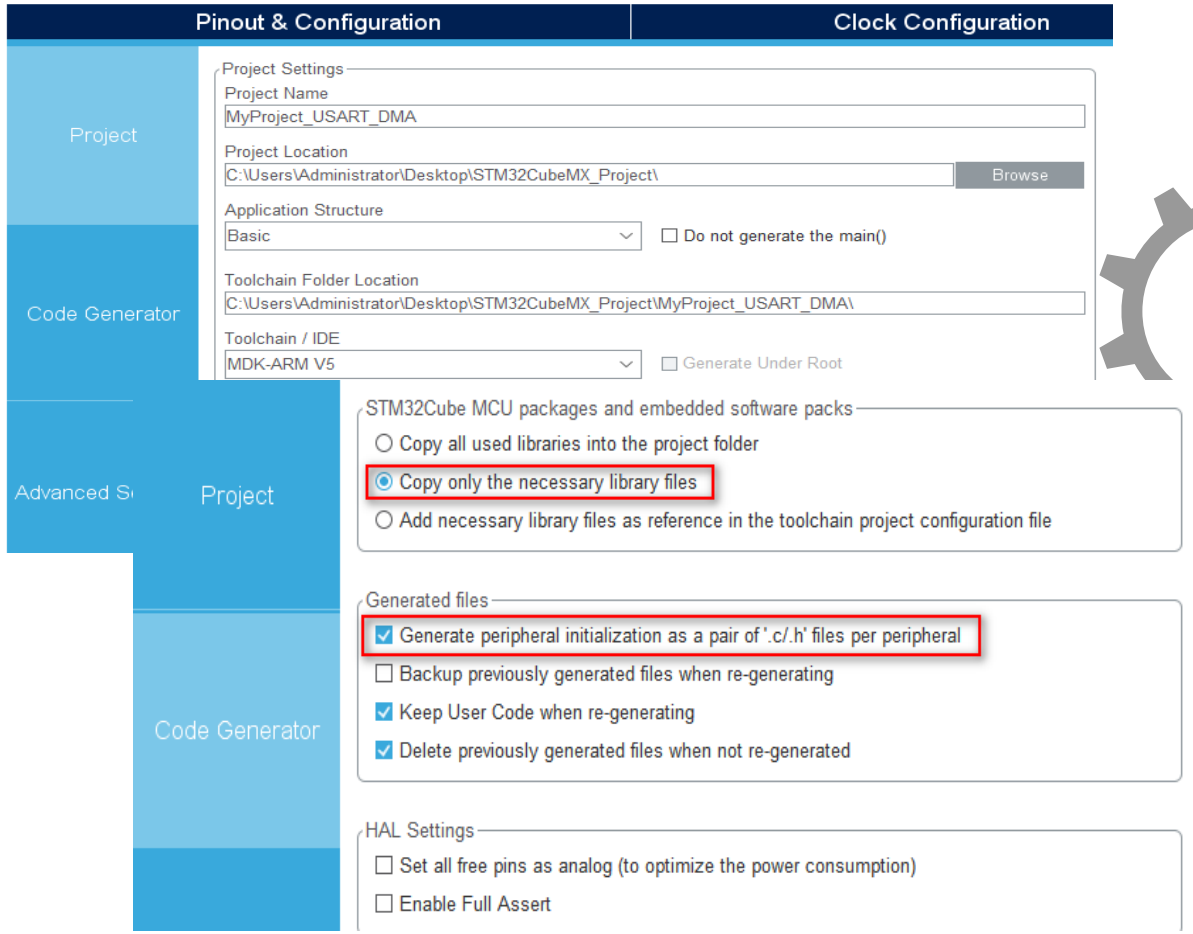


- 在“System Core”中点击“NVIC”，在右侧的配置页面“Configuration”中，可以设置“Priority Group”优先级分组，默认为“4 bits for pre-emption priority 0 bits for subpriority”；
- 勾选“USART1 global interrupt”，“Priority Group”优先级分组可根据具体需要选择合适的分组。

软件设计——生成工程代码

配置keil工程名称和存放位置

“Generate Code”生成工程代码



在main.c文件中添加代码：

- 先定义两个数组DMA_RxBuffer和DMA_TxBuffer，用于接收数据和发送数据
- 将此代码放在main.c文件中的/* USER CODE BEGIN PV */和/* USER CODE END PV */之间。

```
/* Private variables ---------------------*/
/* USER CODE BEGIN PV */
/* Private variables ---------------------*/
uint8_t DMA_RxBuffer[10];
uint8_t DMA_TxBuffer[] = " This is the
sending code with DMA  \r\n ";
/* USER CODE END PV */
```

软件设计——编写应用程序

思考为什么在这里添加？

在int main(void)函数中添加代码：

```
    /* USER CODE BEGIN 2 */
HAL_UART_Receive_DMA(&huart1,DMA_RxBuffer,10);        // 开启DMA接收数据
HAL_UART_Transmit_DMA(&huart1,(uint8_t *)DMA_TxBuffer,sizeof(DMA_TxBuffer));
    /* USER CODE END 2 */
```

```
    /* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  HAL_UART_Transmit_DMA(&huart1,DMA_RxBuffer,10);
  HAL_UART_Receive_DMA(&huart1,DMA_RxBuffer,10);    //重新打开DMA接收数据
}
    /* USER CODE END 4 */
```